UML Use Case Diagram

Identification:

- System: Recursive Sudoku Solver
- Primary actor: User
- Supporting external resource: Puzzle text files
- Primary use case: Solve Sudoku puzzle/puzzles

How it works:

- The user runs the program with the 5 puzzle files or provides a list of filenames. For each file, the system reads the puzzle, displays the original grid, checks the clues, searches recursively, and prints every solution it finds or a message if it cannot solve the puzzle.

Preconditions:

- The program has to be compiled, and the puzzle files must be accessible. Each valid file contains exactly 81 cells, using digits 1-9 or underscores for blanks
    - whitespace is allowed.

Postconditions:

- The program processes every readable puzzle. Each solution it prints follows Sudoku rules and preserves the original clues.
    - Input files are not modified

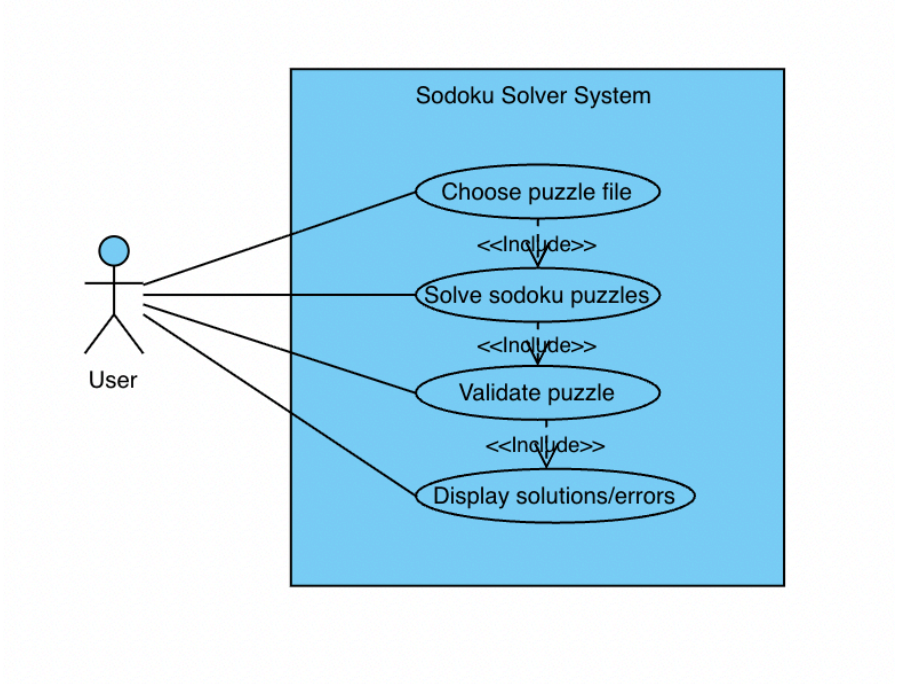
Flow:

- User launches the application (with optional puzzle filenames)
- System selects the supplied filenames, or defaults to puzzle1.txt through puzzle5.txt.
- The system opens a file and attempts to read all 81 cells.
- The system displays the filename and the original 9×9 grid.

    - If a file is missing/ cant be opened, the system reports an input error and continues to the next file.
    - If a file doesnt contain exactly 81 accepted cells or includes unexpected symbols, the system reports an invalid file format and moves on to the next file.
- The system checks that the fixed digits do not conflict with Sudoku rules
- If the clues conflict, the system reports that no solution was found and moves to the next file.
- The system selects an empty cell and tries valid digits from 1 through 9.
- It fills the remaining cells recursively and undoes any choices that lead to a dead end.
- Whenever it fills all 81 cells, the system prints that solution.
- The system keeps searching until it has tried every candidate branch.
- After the search, the system reports the number of solutions and moves on to the next file.
    - If a puzzle has multiple solutions, the system prints each valid board and includes them in the total count.
- If every candidate fails and no solution is found, the system reports “No solution found.”

Use case description:

- Trigger: The user runs Assignment4.exe.
- Goal: Find all solutions for each supplied puzzle
- Success guarantee: The system displays every valid completed board it finds.
- Minimal guarantee: If a puzzle is missing, malformed, contradictory, or unsatisfiable, the system reports the problem without crashing. Related behavior includes selecting files, checking Sudoku rules, and displaying the results.


Sodoku Solver System
Choose puzzle file
<<Include>>
Solve sodoku puzzles
<<Include>>
Validate puzzle
<<Include>>
Display solutions/errors
User